Finding New Data in a Vector Database Split Across Servers: How to Spend a Fixed Network Budget

Panhapiseth Lim
Department of Computer Science, University of Texas Permian Basin, Odessa, Texas, USA
lim_p65274@utpb.edu

***Abstract*—A vector database stores passages as number lists, called vectors, and finds those nearest a query. We simulate one split into 32 parts, or shards. A central coordinator sends each query to two shards, chosen by short summaries of their data. New passages make summaries out of date, so queries can miss new matches. Each time step, a fixed network budget pays for fresh summaries (refresh), extra searches, or copies of new passages (push). Results depend on where new passages land and whether they bring new topics or mix known ones. If each joins the most similar shard, extra search and push beat refresh at default settings. If one shard takes each step's new passages, refresh wins by far on new topics and push on mixed topics. If eight shards share them, push wins on mixed topics. On new topics, neither refresh nor push wins reliably until the largest budget, where push wins. In that setting, refreshing the most-changed shards beat our sample-query probe at budgets of four and eight summaries. Results use one data set, without network delays.**

***Index Terms*—vector database, sharding, query routing, out-of-date summaries, network budget, new data**

I. Introduction

A vector database stores items such as text passages as vectors. A vector is a list of numbers that an AI language model makes from a passage. Think of each vector as a point on a map, where passages with similar meaning sit close together. A search question, or query, becomes a vector the same way. The database returns the stored passages closest to it, which we call the query's best matches.

A distributed vector database splits its data into parts called shards, each kept on its own server. A central server, the coordinator, sends each query to some of the shards and merges their answers. It works like a library with many branch buildings and one front desk. In the system we study, the coordinator sends each query to only two of the 32 shards. To choose them, it keeps one summary per shard: a short list of vectors that mark where its passages lie. A summary is like a catalog card that lists the main topics a branch holds. It picks the two whose summaries come closest to the query.

New passages keep arriving, but the coordinator keeps each shard's old summary until that shard sends a fresh one. An old summary is like an old catalog card that does not list the newest books. So a query may skip the shard that holds its best new match, and miss that match (Fig. 1). This matters for questions about newly added documents, because their answers are in the newest passages.

We study this problem with a simulator, a program that imitates the system one time step at a time. Each run lasts

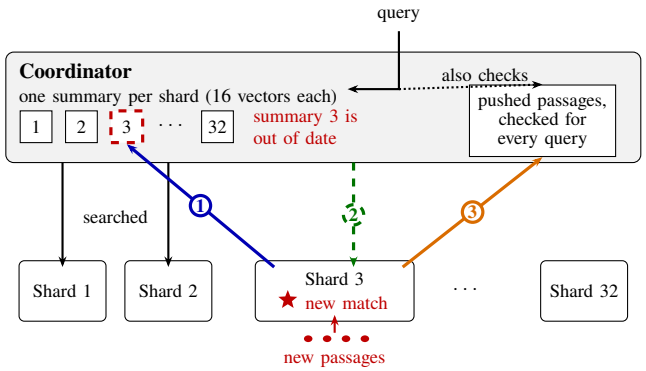


Fig. 1. The problem and the three options. Shard 3 has new passages, but its summary at the coordinator is out of date, so the query can skip shard 3 and miss a new match. Shard 3's summary ranks third closest to the query, so extra search would also ask it.

50 time steps, and about 1,000 new passages arrive per step on average. In each time step, the coordinator also answers a batch of queries, 200 by default. The system may send upkeep messages, paid from a fixed budget of network bytes per time step. The budget works like a monthly phone data plan without rollover: unused bytes are lost. We count it in summaries: our default budget of four pays for four summaries per step. We also test budgets of two and eight. Even the largest is too small to refresh every shard each step. So the question is how to spend the budget, and we compare three options (Fig. 1). The first option is to refresh: some shards send fresh summaries, chosen by a refresh rule. One such rule, most-changed, picks the shards whose data changed most. The second is extra search: when the coordinator is unsure which shards to ask, it also asks a third one. The third is push: shards send copies of their newest passages to the coordinator, which checks them for every query.

We measure accuracy with recall, the share of a query's true best matches that the system returns. For example, if the system returns seven of ten true best matches, its recall is seven in ten. Recent recall counts only the best matches that arrived in the last two time steps. It shows how well the system finds new data, while overall recall counts all best matches, old or new. When we say that a method wins or is better, we mean that it has higher recent recall. A difference is clear when our statistical test shows that chance alone would rarely produce

it. A clear lead is fragile when a further check casts doubt on it (Section IV-A). Our simulator counts the bytes of every message the budget pays for. It checks every answer against the exact best matches.

We vary where new passages land: with the most similar shard, all on one shard, or spread over eight shards. New passages either mix topics the shards already hold or bring new topics, one at a time. We wrote down our plan and tests before the main analysis; Section IV lists its changes.

When each new passage joins the most similar shard, old summaries often still send queries there. Refreshing then gains little, and at the default budget and query rate extra search and push beat every refresh rule. When one shard takes all of a time step's new passages, refresh wins by far on new topics. On mixed topics, fresh summaries barely help with new matches and slightly lower accuracy overall, so push wins. When eight shards share the new passages, push wins on mixed topics. On new topics, refresh has only a fragile lead at a budget of two. Neither refresh nor push clearly wins at four, and push wins at eight.

Refresh raises a second question: which shards should refresh? Our sample-query probe, one of five refresh rules, uses practice queries to estimate which fresh summaries would help most. With eight shards on new topics, it clearly lost to most-changed at budgets of four and eight. The probe's own messages cost it one summary, but most-changed's small score messages were free. So we ran a follow-up in which most-changed pays for them, chosen after seeing the main results and so exploratory. It removed the fragile lead over push. It also showed that at budgets of four and eight, the probe lost mostly by picking worse shards. On average, counting each shard's new passages predicted the value of a fresh summary better than the probe did. We did not try that count as a refresh rule.

II. Related Work

Earlier work has studied which items to fetch or refresh first. Pandey and Olston [1] first fetch the new web pages that would appear most often in the top results for a set of queries. Olston and Widom [2] study how to keep stored copies as fresh as possible when network capacity is too small for exact copies. In their design, each source ranks its own changed items by priority and refreshes those above its own estimate of a shared cutoff. Our sample-query probe adapts these ideas to shard summaries, and we compare refreshing with two other uses of the same budget.

Out-of-date summaries also arise in search systems that send each query to only a few document collections. When the collections change, the summaries used to pick them go out of date [3], [4]. Shokouhi et al. [4] test forty-four changing collections and show that out-of-date collection summaries lower search accuracy. They propose three update policies: update all collections evenly, update more often those that past queries used most, or update by collection size. Three of our refresh rules adapt these policies, and we also try two uses of the budget that refresh nothing.

Jose [5] studies when a vector index, a structure that sorts vectors into groups to speed up search, must regroup them as the data changes. The answer depends on how many groups each query searches, and on how much room that leaves to spare. Slightly outdated group centers, which mark where each group lies, can then be offset by searching a few more groups. That work concerns one index, and our extra search tests a similar trade-off across servers: ask one more shard instead of refreshing summaries.

Finally, some systems spread vector search over many servers. Guo et al. [6] present Manu, a vector database that writes new data into open blocks called segments. A segment is closed to new data once it reaches a size limit or sits idle for some time. Our one-shard placement, where each time step's new data goes to one shard, loosely models such systems. Wang et al. [7] present Semord, where servers work as equals without a central coordinator. To make up for queries sent to the wrong servers, Semord's servers contact extra servers, which resembles our extra search. We instead study a system with a coordinator whose summaries go out of date under a fixed byte budget.

III. How the System Works

A. Data, Shards, and Summaries

We use 100,000 passages and 1,000 test queries from the MS MARCO data set [8]. Its questions come from anonymized user queries, and its passages come from web documents [8]. A language model called MiniLM, from the Sentence-Transformers software package [9], turns each passage and query into a vector of 384 numbers, or 1,536 bytes. We compare two vectors by multiplying their matching numbers and adding the products; a higher score means closer meaning.

The k-means method sorts vectors into k groups of similar vectors and finds the center, or average point, of each group. At setup, k-means from Faiss, a software package for searching and grouping vectors [10], splits the starting passages into the 32 shards. We call the center of a shard's group its starting center. Each shard then runs k-means with 16 groups on its own passages, and its summary is the list of their centers, 24,576 bytes.

To answer a query, the coordinator scores each shard by the summary center closest to the query, and it searches the two best-scoring shards. Each shard stores its vectors in a search graph that links each vector to nearby ones [11]. A search hops along these links toward the query, and it can miss a few close passages. Each searched shard returns the IDs and scores of the 10 closest passages it finds, and the coordinator keeps the best 10. New passages join a shard without moving old ones, and the coordinator keeps the shard's old summary until the shard sends a fresh one.

Each time step draws its queries without repeats from the 1,000 test queries, 200 by default. Under starting-center placement, described below, we also test 50 and 800 queries per time step.

We test two streams, that is, two orders in which new passages arrive. In the mixed-topic stream, half the passages

TABLE I
WHAT EACH OPTION SENDS AND WHAT A BUDGET OF 4 BUYS PER TIME STEP

Item	What is sent	Bytes	Budget of 4 buys
Refresh	One shard's summary (16 centers)	24,576	4 summaries
Extra search	A query to one more shard, and IDs and scores of its 10 best matches	1,656	Up to 59 searches
Push	A new passage and its ID number	1,544	Up to 63 passages
Probe	Its scores, cutoffs, and counts in one time step (Section III-C)	21,632	Room for 3 summaries

build the shards, and the other half arrive in random order, 1,000 per time step. In the new-topic stream, we first sort all passages into 64 clusters of similar passages with k-means, so each cluster is roughly one topic. Half of the clusters, picked at random, build the shards. The other half arrive one cluster after another, about 1,000 passages per time step on average, bringing topics the shards have not seen. This design is modeled on the streaming test of the Big-ANN NeurIPS'23 vector-search competition, which inserts points cluster by cluster [12].

Passages never move once stored, and we test three ways to place new ones. In starting-center placement, each new passage goes to the shard with the closest starting center. In one-shard placement, all of a time step's new passages go to one shard, and the next time step uses the next shard. Every refresh rule then refreshes that shard, because rules serve shards with new passages first (Section III-C). In eight-shard placement, a group of eight shards is open in each time step, and the group moves on by eight shards every step. Each new passage goes to the open shard with the closest starting center, so each step changes five to eight shards. In these last two placements, the coordinator picks shards only by their summaries and does not know which shards are open.

B. The Budget and Its Three Uses

The budget counts the network bytes each time step may use, in units of one summary. We test budgets of two, four, and eight summaries; four is the default and allows 98,304 bytes (Table I). To refresh, a shard rebuilds its summary over all its passages and sends it. Refreshing all 32 shards every time step would cost 786,432 bytes, eight times the default budget.

Extra search spends the budget on the queries where the coordinator is least sure. These are the queries whose second- and third-ranked shards have the closest scores, so either could hold the best matches. For such a query, the coordinator also searches the third-ranked shard. The planned version picks these queries after seeing all of the time step's queries, which a system answering each query on arrival could not do. A budget of four buys up to 59 extra searches, so at 50 queries per step it also asks a fourth shard for some queries. The live version decides as each query arrives and asks at most one extra shard per query. It asks when the score gap is below a cutoff: the gap that would have spent the budget exactly in the previous time step. With push, shards send up to the budget's worth of their newest passages each time step. The coordinator keeps them all and checks each query against them.

We also compare against two reference methods that ignore the budget. Never refresh keeps the first summaries, and refresh all sends all 32 fresh summaries every time step. Refresh all is a reference, not the best that refresh can do. Its rebuilt summaries still have only 16 centers, so they describe a shard roughly. Searching all 32 shards finds more than 99.4% of recent matches, so almost all misses come from choosing shards.

C. Five Refresh Rules

A refresh rule picks which shards refresh in each time step. Every rule serves first the shards with new passages since their last refresh, putting any that waited too long ahead. The waiting limit is twice the time a rule needs to refresh every shard once. That is 16 time steps for a rule that refreshes four summaries per step. Any budget left over re-sends the summaries of unchanged shards. Rules that score shards break ties by the lowest shard number.

Rotation lets shards take turns. Most-searched favors the shards that past test queries were sent to most often, and largest-shard favors the shards with the most passages. Their scores do not reset after a refresh, so they keep picking the same shards until the waiting limit steps in. Most-changed scores a shard by the share of its passages that are new. It multiplies that share by how far the new passages lie, on average, from the nearest center in the shard's last summary. Each shard computes this score itself, so a real system would send one 4-byte score per shard, or 128 bytes per time step. Our main runs did not charge these bytes; the follow-up in Section V-E does.

The sample-query probe is our own rule and the most complex. It adapts two ideas from Section II: ranking by how often items reach top results [1], and letting each source rank its own changes [2]. The probe uses 64 sample queries: practice questions from a separate part of the data set, not the test queries. The coordinator sends them to every shard once at setup. Each time step then has three rounds of messages. First, for each sample query, the coordinator asks the two shards that its summaries rank highest for the scores of their 10 best matches. Second, it sends every shard two cutoffs for each sample query. One is the summary score a shard would need to rank in the top two. The other is the score of the query's 10th-best match so far. Third, each shard with new passages rebuilds its summary locally. For each sample query, it checks whether the rebuilt summary would lift it into the top two. For those queries, it counts its new passages that score above the 10th-best cutoff and sends that count. Among shards with new passages, those over the waiting limit refresh first. The rest go in order of their counts, with ties going to the lowest shard number.

These rounds cost 21,632 bytes per time step. Each budget is a whole number of summaries, and refresh can send only whole summaries. So any extra charged cost, however small, uses up the room for one summary. At a budget of four, the

probe therefore refreshes three summaries where the other rules refresh four, and its waiting limit is 22 time steps.

D. Costs We Do Not Count

The budget pays only for the messages in Table I, and six other costs are left out. The first is sending new passages to the shards, which is the same for every method. The second is the probe's setup broadcast of 3,145,728 bytes, its small per-step lists of which sample queries to answer, and its shard work. Every changed shard rebuilds a summary just to compute its count, and the shards run 128 sample-query searches per time step. Spread over a run, the broadcast would add 62,915 bytes per step. That is more than a budget of two, and at four it would leave no room for any summary. The third is most-changed's score bytes, in the main runs. The fourth is a free flag that tells the coordinator which shards have new passages. The flag says only that a shard has new passages, not what they are about. Every refresh rule uses it to choose shards to refresh, but queries are still sent by summaries alone. Largest-shard also gets the shard sizes for free. Charging even a few bytes for this flag would cost every refresh rule one whole summary per time step. The fifth is the added work of extra search: at the default query rate, shards run 7.2% to 29.5% more searches across budgets. At a budget of four and 50 queries per step, they run up to 59.0% more. Asking a third shard also means contacting three shards at once or waiting for a second exchange of messages, and neither is counted. The sixth is push's memory and scanning, up to 3,150 kept passages at a budget of four and 6,350 at eight. Scanning them for each query takes 6.2 to 12.4 times as many vector comparisons as scoring all summaries. Keeping only the last two time steps' 126 would give the same recent recall at a budget of four. Push also assumes the shards know which passages arrived last overall, since it sends the newest ones. Leaving out the probe's extra costs, most-changed's score bytes, and the free flag favors the refresh rules. Leaving out the extra search work and push's memory and scanning favors extra search and push.

IV. How We Tested

A. Measuring and Comparing Accuracy

In every time step, an exact search over all stored passages finds each test query's true 10 best matches and when each was added. *Recent recall* is the share of a query's true best matches from the current and previous time steps that the system returns. Only queries with at least one recent match count toward it: about 24% of the mixed-topic queries and 13% of the new-topic queries. *Overall recall* counts all 10 true best matches, old or new. We give recall as a percentage and call a difference of one percentage point one *point*.

We run each main setting five times, with seeds numbered 42 to 46. A seed is a number that fixes a run's random choices, the way a recorded shuffle fixes the order of a deck of cards. It sets the arrival order, the clusters, the queries asked, and the shards' search graphs. The passages are the same in every run, and all methods see the same queries, so we compare methods query by query.

To compare two methods, we average the difference in recall between them over all queries and seeds. Other seeds or queries would give a somewhat different average, so we ask how far it could move by chance. Our main test is a 95% range found by resampling. We redraw the five seeds with repeats allowed, like drawing names from a hat and putting each one back. Within each drawn seed, we redraw its queries the same way. We recompute the average difference for each of 2,000 such redraws, and the middle 95% of these averages is the range. If the whole range lies on one side of zero, chance alone would rarely produce the difference, and we call it *clear*. Otherwise we say there is no clear difference.

Our simpler test uses only the five seed averages of the difference. Their *spread* is their standard deviation, roughly the typical distance of one seed's average from the mean of all five. This textbook range is centered on that mean, and its half-width is about 2.8 times the spread divided by the square root of five. The large factor allows for having only five numbers. The two tests can disagree because resampling also counts how single queries vary, but with only five seeds it understates how much seeds differ. So when seeds differ a lot, as in several main eight-shard tests in Table II, our range was 19% to 27% narrower. When the five seed averages agree closely, the simpler range shrinks and ours can be wider: 51% wider for the probe against most-searched (Section V-D).

A clear lead is *fragile* if it fails any one of four further checks. Its range must not nearly touch zero, and the simpler test must exclude zero. No seed may point the other way, and a most-changed lead must survive paying for its score messages (Section V-E).

Our ranges treat each asked query as a separate, unrelated sample, although the same 1,000 queries repeat about 10 times per run. They also rest on the one sample of passages and queries we drew from the data set. Nor are they widened for our many comparisons, which raises the chance that some clear result is a fluke. Against other options we use the *best refresh rule*: whichever of the five rules has the highest recent recall in that setting. Picking it from the same runs we compare favors refresh a little. This makes the alternatives' tests harder to pass under starting-center placement, and refresh's tests slightly easier under eight-shard placement.

B. Our Plan, Written Before the Main Analysis

Writing tests down before seeing the results keeps researchers from reporting only the tests that happen to look good. Before any run, we stored a plan with first settings, message sizes, and three hypotheses, or predictions. One was that the probe would beat the other rules by at least 5 points under starting-center placement. A small trial run, or pilot, with one seed (42) then showed that refresh gained little there, and all three were replaced.

Before the main runs, a code review and a first written change to the plan, called an addendum, revised the design. Together they added the new-topic stream, one-shard placement, the three query rates, and an ID number in each pushed passage. They also replaced a rule that slowly raised waiting

shards' scores with a waiting limit set by the budget. The first addendum set five of the tests below. Two of them restate pilot results and reuse the pilot's seed, so Section V-A also checks them on the other four seeds. Testing our analysis program on a partial set of 117 starting-center runs printed one result, which changed no test or rule. A second addendum came when 140 of the 330 one-shard runs had finished, before anyone looked at them and before any eight-shard run. It added eight-shard placement, live extra search, and the remaining tests, which is why one-shard runs have only planned extra search.

We call the tests in the plan registered tests; a test that one method beats another passes only if the whole range is above zero. Under starting-center placement, we tested that refresh all gains less than 5 points on average over never refresh. We also tested that planned extra search, live extra search, and push each beat the best refresh rule. Under one-shard placement, we predicted that all five refresh rules match refresh all exactly. We also tested that refresh beats extra search and push at every budget. The probe test asked whether the probe beats each other rule by at least 5 points on average, with the whole range above zero. It ran only if refresh all first gained at least 5 points on average, which we call the 5-point threshold. Under one-shard placement, the predicted tie meant that this test could not pass. Eight-shard placement repeats these tests with live extra search. It adds a prediction that the four rules other than the probe match refresh all at a budget of eight.

After reading all the main results, we wrote down a follow-up and stored it before running it. In it, most-changed pays for its score bytes, so it refreshes as many summaries as the probe; we call this version paying most-changed. Of its 42 simulations, 30 used eight-shard placement. Another 10 used starting-center placement at 800 queries per step. The last 2 were reruns with the charge turned off, and they matched the stored main runs on every query. Because we chose the follow-up after seeing the results, it is exploratory and has no pass or fail tests. It has 64 ranges, one for each of 32 comparisons on recent and on overall recall, not widened for many comparisons. Table II shows the eight-shard ones against the probe, push, and live extra search, on recent recall.

The main experiments have 1,116 runs, with none missing, and every run stayed within its budget for the costs we charge. Runs that ran out of memory were rerun from the same code. They include some starting-center runs, two eight-shard runs, and 22 of the 30 eight-shard follow-up runs.

V. Results

Table II gives the main differences, and Fig. 2 shows recent recall by budget; results use 200 queries per time step unless we say otherwise.

A. Starting-Center Placement: Old Summaries Often Still Work

When each new passage joins the shard with the closest starting center, the old summaries often already point queries toward the right shard. So refresh all, at eight times the default budget, raised recent recall by only 0.5 points on the mixed-topic stream, which is no clear difference. On the new-topic stream it gained 2.3 points, and the registered test that the gain stays below 5 points passed on both streams. Even the top ends of both ranges were below it. Better shard choice could still help a lot. Suppose each query went to the two shards holding the most of its 10 true best matches, old or new. On the mixed-topic stream, that finds 80.0% of recent matches, against 70.8% for refresh all.

At the default budget of four, planned extra search, live extra search, and push each clearly beat every refresh rule on both streams. Planned extra search and push even beat refresh all, while no refresh rule clearly beat never refresh on the mixed-topic stream. On each of the four seeds the pilot did not use, planned extra search and push still beat the best refresh rule on both streams.

Extra search helps the coordinator pick good shards in general, not only for new passages. It also raised overall recall over the best refresh rule by 2.1 to 3.0 points. Push, in contrast, helps only through the new passages it keeps: 6.30% of them at a budget of four on the mixed-topic stream. Never refresh found 70.3% of recent matches there. If push simply found that same share of the matches never refresh missed, it would reach 72.1%, and it reached 72.3%. Push raised overall recall over the best refresh rule by only 0.3 to 0.5 points.

At 50 and 800 queries per time step, extra search and push still beat every refresh rule on the mixed-topic stream. On the new-topic stream at 800 queries, most-changed overtook both versions of extra search. Its lead was 1.0 to 1.1 points while its score bytes were free, and it shrinks when they are charged (Section V-E). Push still beat refresh at every query rate. From 200 queries up, the planned version of extra search was at most 0.21 points ahead of the live one. At the lowest query rate, it led by 1.3 to 2.0 points. That is partly because the live version asks at most one extra shard per query and spent only about 83% of the planned version's bytes.

B. One-Shard Placement: The Topics Decide

When one shard takes all of a time step's new passages, all five refresh rules matched refresh all on every query, as predicted. So the registered probe test could not pass on the new-topic stream. It was skipped on the mixed-topic stream, where refresh all missed the 5-point threshold. Since only one shard changes, the four rules other than the probe spend the rest of each budget re-sending unchanged summaries. That is 50% of a budget of two and 87.5% of a budget of eight.

On the mixed-topic stream, never refresh finds 6.3% of recent matches, about what chance gives when two of 32 shards are searched. Refresh all raises this only to 7.9%, and it lowers overall recall by 1.6 points, with a drop in every seed. Plausibly, 16 centers cannot describe a shard that holds a mix of every topic. Push wins at every budget (Fig. 2a), so the registered test that refresh beats both alternatives failed on this stream. Push keeps 3.10% to 12.70% of new passages across budgets, and its gain over never refresh closely matches what that share predicts.

TABLE II
MAIN RESULTS AT 200 QUERIES PER TIME STEP: RECENT RECALL OF THE FIRST METHOD MINUS THE SECOND, IN POINTS

Difference: first method minus second; positive favors the first. Clear winner: the favored method if the 95% range excludes zero, else neither.

Placement	Budget	First method vs. second method	Mixed-topic stream Difference [range]	Clear winner	New-topic stream Difference [range]	Clear winner
Starting-center	4	Refresh all vs. never refresh	+0.55 [−0.07, +1.12]	Neither	+2.29 [+1.47, +3.17]	Refresh all
	4	Best refresh rule vs. planned extra search	−2.76 [−3.22, −2.33]	Planned extra search	−1.97 [−3.15, −0.74]	Planned extra search
	4	Best refresh rule vs. live extra search	−2.55 [−2.97, −2.17]	Live extra search	−1.84 [−3.01, −0.58]	Live extra search
	4	Best refresh rule vs. push	−1.88 [−2.36, −1.44]	Push	−2.09 [−3.27, −1.00]	Push
One-shard	4	Refresh all vs. never refresh	+1.66 [+1.17, +2.24]	Refresh all	+22.61 [+20.83, +24.40]	Refresh all
	2	Any refresh rule vs. push	−1.40 [−1.96, −0.82]	Push	+19.40 [+17.67, +21.12]	Refresh
	4	Any refresh rule vs. push	−4.31 [−5.09, −3.47]	Push	+16.04 [+14.40, +17.73]	Refresh
	8	Any refresh rule vs. push	−10.15 [−11.02, −9.23]	Push	+9.64 [+8.04, +11.26]	Refresh
Eight-shard	4	Refresh all vs. never refresh	+1.16 [+0.38, +1.91]	Refresh all	+6.74 [+5.16, +8.16]	Refresh all
	2	Best refresh rule vs. push	−1.79 [−2.38, −1.20]	Push	+1.47 [+0.03, +2.90]	Refresh, fragile
	4	Best refresh rule vs. push	−3.67 [−4.57, −2.85]	Push	+1.04 [−0.50, +2.53]	Neither
	8	Best refresh rule vs. push	−7.17 [−8.00, −6.31]	Push	−3.59 [−5.37, −1.86]	Push
	2	Best refresh rule vs. live extra search	−1.08 [−1.54, −0.62]	Live extra search	+2.75 [+0.97, +4.33]	Refresh, fragile
	4	Best refresh rule vs. live extra search	−2.09 [−2.84, −1.33]	Live extra search	+3.86 [+2.10, +5.31]	Refresh
	8	Best refresh rule vs. live extra search	−4.28 [−5.17, −3.43]	Live extra search	+1.47 [+0.07, +2.81]	Refresh, fragile
	4	Most-changed vs. probe	+0.39 [−0.32, +1.06]	Neither	+3.60 [+2.26, +4.93]	Most-changed
Exploratory follow-up (Section V-E): most-changed pays for its score bytes, so it refreshes as many summaries as the probe						
Eight-shard	2	Paying most-changed vs. probe	+0.30 [−0.29, +0.88]	Neither	+0.48 [−0.48, +1.66]	Neither
	4	Paying most-changed vs. probe	+0.19 [−0.29, +0.63]	Neither	+2.66 [+1.46, +4.00]	Paying most-changed
	8	Paying most-changed vs. probe	+0.19 [−0.29, +0.64]	Neither	+1.76 [+1.04, +2.59]	Paying most-changed
	2	Paying most-changed vs. push	−1.58 [−2.11, −1.06]	Push	−0.16 [−1.89, +1.49]	Neither
	4	Paying most-changed vs. push	−3.87 [−4.72, −3.11]	Push	+0.10 [−1.40, +1.52]	Neither
	8	Paying most-changed vs. push	−7.16 [−7.98, −6.33]	Push	−3.42 [−5.15, −1.75]	Push
	2	Paying most-changed vs. live extra search	−0.87 [−1.41, −0.36]	Live extra search	+1.12 [−0.39, +2.62]	Neither
	4	Paying most-changed vs. live extra search	−2.29 [−2.90, −1.68]	Live extra search	+2.92 [+1.39, +4.26]	Refresh
	8	Paying most-changed vs. live extra search	−4.28 [−5.17, −3.42]	Live extra search	+1.63 [+0.27, +2.95]	Refresh, fragile

Fragile: clear, but failing a further check (Section IV-A). Best refresh rule: starting-center, the probe on mixed topics and most-changed on new ones; eight-shard, largest-shard at a budget of two on mixed topics, the four tied rules other than the probe at eight, and most-changed otherwise.

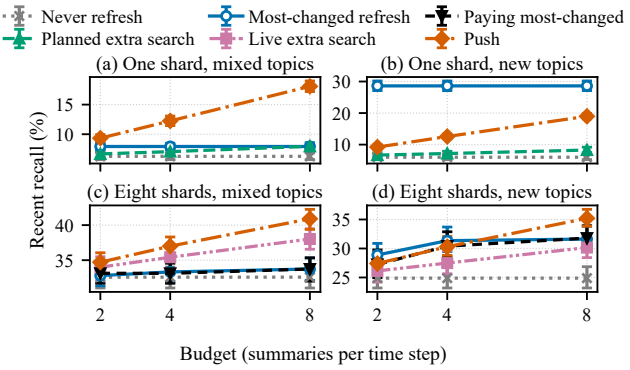


Fig. 2. Recent recall at each budget, at 200 queries per time step. Top: one-shard placement, where all refresh rules tie and only planned extra search was run. Bottom: eight-shard placement, with live extra search and paying most-changed (Section V-E); most-changed is the best refresh rule there, except that largest-shard is slightly higher at a budget of two on mixed topics. Vertical lines show each method's own 95% range. Table II compares methods query by query, so it can show a clear difference even where these ranges overlap. Vertical scales differ.

On the new-topic stream, the result reverses. There, each time step's passages come from one or two new clusters, which 16 centers describe well. Refresh all raises recent recall from 6.0% to 28.6%. Refresh beats push at every budget, but its lead shrinks from 19.4 points at a budget of two to 9.6 at eight. Larger budgets push more passages, while 1 fresh summary per time step is already enough. Refresh also beats planned extra search at every budget, by 20.3 to 21.9 points, so the registered test passed on this stream.

C. Eight-Shard Placement: Close Calls

When eight shards share each time step's new passages, more shards change than the smaller budgets can refresh. On the mixed-topic stream, refresh all again gains little, 1.2 points, and lowers overall recall by 0.9 points, with a drop in every seed. Push beat refresh at every budget, and so did live extra search (Fig. 2c). On the new-topic stream, refresh all gains 6.7 points and passes the 5-point threshold on average. In one seed, though, it gained only 4.1 points. The best two shards per query, chosen as in Section V-A, would find 50.5% of recent matches on mixed topics, against 33.7% for refresh all. On new topics they would find 48.6%, against 31.6%.

On new topics at a budget of two, most-changed, the best refresh rule, leads push by 1.5 points, but the lead is fragile. Its range starts at only 0.03 points, one seed favors push, the simpler test includes zero, and it relies on free score bytes (Section V-E). At a budget of four neither clearly wins, and at eight push wins by 3.6 points (Fig. 2d). Most-changed also leads live extra search on new topics at every budget, but only the lead at four is not fragile. At two, one seed favors live extra search and the lead relies on free score bytes; at eight, the simpler test includes zero. So the registered test that refresh beats both alternatives failed at every budget on mixed topics. On new topics it passed only at a budget of two, and that pass is fragile.

On overall recall, which also counts older matches, live extra search beat push at every budget, on both streams, by 0.5 to 2.5 points. It also beat refresh there on new topics at budgets of four and eight.

On new topics, push beats never refresh by about 0.2 to 1.0 points more than its share of new passages predicts. Plausibly, this is because push keeps each time step's last passages, which on this stream tend to come from the newest cluster. The planned version of extra search was at most 0.30 points ahead of the live one here. The live version spent slightly less than its budget, as little as 88.7% of it. That shortfall, worth only about 0.13 to 0.18 points, changes no winner but blurs how much of the planned lead comes from planning ahead.

D. Which Shards Should Refresh?

We expected the probe to beat each other rule by at least 5 points, but it did not. In eight-shard placement on new topics at a budget of four, most-changed found 3.6 points more recent matches than the probe (Table II). The probe showed no clear difference from most-searched or largest-shard. It clearly beat only rotation, which itself did no better than never refresh at budgets of two and four. Against most-searched, all five seeds and the simpler test favored the probe, but our range, wider here (Section IV-A), included zero. On mixed topics the probe test was skipped, because refresh all missed the 5-point threshold. At a budget of eight, the four rules other than the probe refreshed all eight open shards, matching refresh all on every query as predicted. The probe refreshed seven and on average trailed refresh all by 1.6 points on new topics. On mixed topics it trailed by 0.2 points, with no clear difference.

The probe's estimate was often zero for every shard. That happens when no sample query would both move a shard into its top two and find a new passage among its 10 best matches. In eight-shard placement at a budget of four, this happened in 24.8% of time steps on new topics and 48.4% on mixed topics. Under starting-center placement on the mixed-topic stream, its estimate was zero for every shard in 66.8% of time steps. The probe then falls back to shards over the waiting limit first, then the lowest shard number.

To see which signal points to the shards worth refreshing, we measure each shard's benefit in each time step. The benefit counts the test queries' true best matches that would move into the two searched shards if that shard alone sent a fresh summary. It counts old and new matches and does not subtract matches that would drop out. We rank the 32 shards by this benefit and by a signal, and compare the two rankings with Spearman's rank correlation. Like two judges ranking the same contestants, it is one when they agree exactly, near zero when unrelated, and minus one when reversed. We average it over the time steps in which both the benefit and the probe's estimate differ between shards, and then over the five seeds. Skipping the steps where the probe cannot tell shards apart favors the probe, and the averages have no ranges. In eight-shard placement on new topics at a budget of four, the probe's estimate scored 0.26, measured on the runs that used the probe. A plain count of each shard's new passages scored 0.63. Unlike most-changed, the count ignores how far the new passages lie from the old centers, and we did not measure most-changed this way. On average, the count scored higher in every starting-center and eight-shard setting. It is free only if the coordinator tracks how many passages each shard receives.

E. Follow-Up: When Most-Changed Pays for Its Scores

The follow-up charges most-changed for its own score messages. Paying 128 bytes per time step costs it a whole summary and leaves 24,448 bytes of each step's budget unused. This is a pessimistic case for most-changed, because unused bytes do not carry over, and we did not run a version where they do. If unused bytes carried over, its 128 bytes per step would add up to less than one summary over a 50-step run.

Paying most-changed refreshes as many summaries as the probe, so comparing the two shows which picks better shards. Of most-changed's 3.6-point lead over the probe in the main runs, at a budget of four on new topics, 2.7 points remained. The other 0.9 points came from the summary that the probe's messages cost it. At a budget of eight, paying most-changed led by 1.8 points. All 10 seed results at these two budgets favored it. At a budget of two on new topics, 1.6 of the probe's 2.1-point main-run loss came from the lost summary. That part is fragile, because the simpler test includes zero. At that budget, and on mixed topics, the two rules showed no clear difference. This comparison still favors the probe, which sends 21,504 more bytes per time step and keeps its setup broadcast free. At a budget of four on new topics, paying most-changed led by 5.4 points in steps where the probe's estimate was all zero. In the other steps it led by 1.9. There, the waiting limit forced 100 of the probe's 750 refreshes. It forced 16 of the 750 made by paying most-changed.

Against push on new topics, paying most-changed lost the fragile lead that most-changed had at a budget of two. Push was then 0.16 points ahead, with no clear difference, so this is not a win for push either. At a budget of four there was still no clear winner, and at eight push led by 3.4 points. Against live extra search on new topics, paying most-changed had no clear lead at a budget of two and a clear lead at four. At eight its lead was fragile, because one seed pointed the other way. On overall recall, live extra search led it on new topics at budgets of four and eight, though at four the simpler test includes zero. On mixed topics, push and live extra search each led paying most-changed at every budget. At a budget of eight, paying most-changed refreshes seven summaries, so it no longer matches refresh all on every query. On new topics it differs from unpaid most-changed by only 0.16 points, with no clear difference.

The follow-up also reran starting-center placement at 800 queries per step, where most-changed had overtaken extra search on new topics. Paying most-changed still edged planned extra search there, by only 0.38 points. That lead is fragile: its range barely clears zero, and it touches zero when we redo the resampling with other random draws. It also comes only from the first 30 of the 50 time steps. Planned extra search was 0.20 points ahead in later steps and 0.20 ahead on overall recall, with no clear difference there. Paying most-changed also led live extra search by 0.43 points, with a range starting at 0.07. Push still beats paying most-changed there, by 2.4 points.

VI. Limits and What We Did Not Test

Our simulator runs as a single program, so all shards share one machine. It does not model network delays, waiting in queues, work done at the same time, or failures. All runs use one data set and one text model, with 100,000 passages drawn once with a fixed seed. They also use five seeds and fixed sizes: 32 shards, two searched shards per query, and 16 centers per summary.

About 1,000 passages arrive per time step, and a summary stays the same size as arrivals grow. But the share of new passages that push can keep at a fixed budget falls, so higher arrival rates may favor refresh. In one-shard and eight-shard placement, new passages join shards that keep their starting passages, and a fresh summary covers the whole shard. On mixed topics this lowered overall recall, plausibly because the centers shift toward the random new passages. Separate summaries for new data might avoid this, but we did not model them. A coordinator that knows which shards are open could also reach them without upkeep traffic. Saving re-sent summary bytes, carrying unused bytes over, and a refresh rule based on the plain count are also untested.

VII. Conclusion

When shard summaries go out of date, a split vector database can miss new matches. The best use of a fixed network budget for finding them depends on where new data goes and its topics.

In our tested setting, the results suggest three rules of thumb. First, when new data joins the most similar shard, spend the budget on extra search if shards have spare capacity. Push is the other choice, if the coordinator can store and scan new passages. Extra search won at every query rate on mixed topics and up to 200 queries per step on new topics. At 800 queries on new topics, most-changed overtook it, but only fragilely once it paid for its scores. Push won at every rate we tried, but at the default rate extra search raised overall recall far more. Second, when each step's new data lands on one shard, refresh that shard if the data brings a new topic. When topics are mixed, push instead, since fresh summaries there also lower overall recall. When eight shards share the new data, push on mixed topics. On a new topic, neither refresh nor push wins reliably at the smaller budgets, and push wins at the largest. Third, to choose which shards to refresh, rank them by how much their data changed rather than by probing. Our evidence comes from eight shards on new topics at budgets of four and eight, even with scores paid. It also rests on recall alone, since we did not score most-changed with our ranking measure.

These results come from one simulator and one data set, with some costs uncharged. Future work should test summaries that adapt to new data and coordinators that know where new data goes. It should also test higher arrival rates and slow responses.

Acknowledgment

Gemini (Google) drafted the simulator, analysis code, and number scripts, and an earlier draft of the text. Claude (Anthropic) rewrote that draft into this version, wrote the figure code, added scripted number commands, checked each citation against its source, and recomputed reported numbers. Gemini, Claude, and Codex (OpenAI) reviewed drafts; Codex edited no files. Result numbers come from scripts run on stored run outputs. The author takes responsibility for the content.

References

[1] S. Pandey and C. Olston, "Crawl ordering by search impact," in *Proceedings of the International Conference on Web Search and Web Data Mining (WSDM '08)*, 2008, pp. 3–14. [Online]. Available: https://doi.org/10.1145/1341531.1341535

[2] C. Olston and J. Widom, "Best-effort cache synchronization with source cooperation," in *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data (SIGMOD '02)*, 2002, pp. 73–84. [Online]. Available: https://doi.org/10.1145/564691.564701

[3] P. G. Ipeirotis, A. Ntoulas, J. Cho, and L. Gravano, "Modeling and managing content changes in text databases," in *21st International Conference on Data Engineering (ICDE '05)*, 2005, pp. 606–617. [Online]. Available: https://doi.org/10.1109/ICDE.2005.91

[4] M. Shokouhi, M. Baillie, and L. Azzopardi, "Updating collection representations for federated search," in *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '07)*, Amsterdam, The Netherlands, 2007, pp. 511–518. [Online]. Available: https://eprints.gla.ac.uk/3860/1/shokouhi3860.pdf

[5] J. Jose, "When does a partitioned ANN index need active re-partitioning under drift? A characterization and benchmark," Research Square preprint, version 1, 2026. [Online]. Available: https://doi.org/10.21203/rs.3.rs-9916583/v1

[6] R. Guo, X. Luan, L. Xiang, X. Yan, X. Yi, J. Luo, Q. Cheng, W. Xu, J. Luo, F. Liu, Z. Cao, Y. Qiao, T. Wang, B. Tang, and C. Xie, "Manu: A cloud native vector database management system," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3548–3561, 2022. [Online]. Available: https://doi.org/10.14778/3554821.3554843

[7] S. Wang, Y. Liu, Y. Hua, X. Zhang, and C. Qian, "Semord: Learned semantic-preserving placement and low-fanout routing for distributed vector search," arXiv preprint arXiv:2609.25514, 2026. [Online]. Available: https://arxiv.org/abs/2609.25514

[8] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A human generated MAchine Reading COmprehension dataset," in *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016)*, ser. CEUR Workshop Proceedings, vol. 1773, 2016, arXiv preprint arXiv:1611.09268. [Online]. Available: https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992. [Online]. Available: https://doi.org/10.18653/v1/D19-1410

[10] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," arXiv preprint arXiv:2401.08281, 2024. [Online]. Available: https://arxiv.org/abs/2401.08281

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. [Online]. Available: https://arxiv.org/abs/1603.09320

[12] H. V. Simhadri, M. Aumüller, A. Ingber, M. Douze, G. Williams, M. D. Manohar, D. Baranchuk, E. Liberty, F. Liu, B. Landrum, M. Karjikar, L. Dhulipala, M. Chen, Y. Chen, R. Ma, K. Zhang, Y. Cai, J. Shi, Y. Chen, W. Zheng, Z. Wang, J. Yin, and B. Huang, "Results of the Big ANN: NeurIPS'23 competition," arXiv preprint arXiv:2409.17424, 2024. [Online]. Available: https://arxiv.org/abs/2409.17424